

SoundSphere Master Project Guide

Comprehensive Version-wise Development Blueprint

1. Project Vision

SoundSphere is a social music platform that sits on top of existing streaming services rather than replacing them. Its mission is to let people enjoy synchronized music sessions, social interaction and communities regardless of the streaming provider they use.

2. Core Problems

- Friends use different streaming platforms.
- Existing listen-together features are limited.
- There is no dedicated social network centered around music.
- Remote friends and couples lack immersive shared experiences.

3. Version-wise Roadmap

Version 1 – Foundation (Authentication & Platform Setup)

Goal

Purpose: Build a stable user identity layer and streaming integration. No rooms or chat yet.

Features

- Email/Google login
- User profile
- Connect Spotify
- Basic dashboard
- Friend invitations

Development Steps

1. Design UI.
2. Design database tables.
3. Build backend APIs.
4. Integrate frontend.
5. Perform testing.
6. Release and gather feedback.

Advantages

Advantages: Smaller scope, faster delivery, easier testing.

Disadvantages

Disadvantage: Users cannot yet experience the main value.

How to Convert the Disadvantage into an Advantage

Convert to Advantage: Release quickly, collect feedback and validate onboarding.

Version 2 – Social Listening MVP

Goal

Purpose: Deliver the first complete listening experience.

Features

- Private rooms
- Invite friends
- Playback synchronization
- Queue management
- Song voting
- Session history

Development Steps

1. Design UI.
2. Design database tables.
3. Build backend APIs.
4. Integrate frontend.
5. Perform testing.
6. Release and gather feedback.

Advantages

Advantages: Validates the core business idea.

Disadvantages

Disadvantage: No live communication.

How to Convert the Disadvantage into an Advantage

Convert to Advantage: Keep implementation simple while perfecting synchronization.

Version 3 – Communication & Communities

Goal

Purpose: Transform the app from a listening tool into a social platform.

Features

- Real-time chat
- Voice chat
- Emoji/GIF reactions
- Public rooms
- Communities
- Moderation tools
- Notifications

Development Steps

1. Design UI.
2. Design database tables.
3. Build backend APIs.
4. Integrate frontend.
5. Perform testing.
6. Release and gather feedback.

Advantages

Advantages: Higher engagement and retention.

Disadvantages

Disadvantage: Higher infrastructure cost and moderation challenges.

How to Convert the Disadvantage into an Advantage

Convert to Advantage: Use Socket.IO, moderation rules, reporting, and scalable real-time architecture.

Version 4 – AI & Premium

Goal

Purpose: Differentiate SoundSphere with intelligence and monetization.

Features

- AI DJ
- AI playlists
- Music memories
- Mood detection
- Premium plans
- Creator rooms
- Cross-platform expansion

Development Steps

1. Design UI.
2. Design database tables.
3. Build backend APIs.
4. Integrate frontend.
5. Perform testing.
6. Release and gather feedback.

Advantages

Advantages: Strong competitive advantage and revenue.

Disadvantages

Disadvantage: AI/API cost and dependency.

How to Convert the Disadvantage into an Advantage

Convert to Advantage: Cache results, optimize prompts, offer premium AI features.

4. Suggested Database

- Users table with appropriate foreign-key relationships.
- StreamingAccounts table with appropriate foreign-key relationships.
- Rooms table with appropriate foreign-key relationships.
- RoomMembers table with appropriate foreign-key relationships.
- Songs table with appropriate foreign-key relationships.
- Queue table with appropriate foreign-key relationships.
- Votes table with appropriate foreign-key relationships.
- Messages table with appropriate foreign-key relationships.
- Communities table with appropriate foreign-key relationships.
- Notifications table with appropriate foreign-key relationships.

5. Suggested Backend APIs

POST /auth/login

POST /rooms

POST /rooms/{id}/join

POST /queue/add

POST /vote

GET /messages

POST /messages

POST /voice/token

GET /notifications

6. Recommended Tech Stack

Frontend: Next.js + React + Tailwind CSS

Backend: Node.js + Express

Database: PostgreSQL + Redis

Realtime: Socket.IO

Voice: WebRTC

Hosting: Vercel + Railway

Storage: Supabase or S3

7. Risks & Mitigations

API restrictions: Use official SDKs and graceful fallbacks.

Synchronization: Implement latency compensation and periodic resync.

Moderation: Reporting, blocking and community guidelines.

Scalability: Redis caching and horizontal scaling.

8. Final Recommendation

Focus on one version at a time. Do not build every feature at once. Perfect Version 2 (the core listening experience) before adding chat in Version 3 and AI in Version 4. This phased strategy minimizes risk, improves quality, and makes the product attractive to users and investors.